



BOHIO SYSTEMS × KINAN

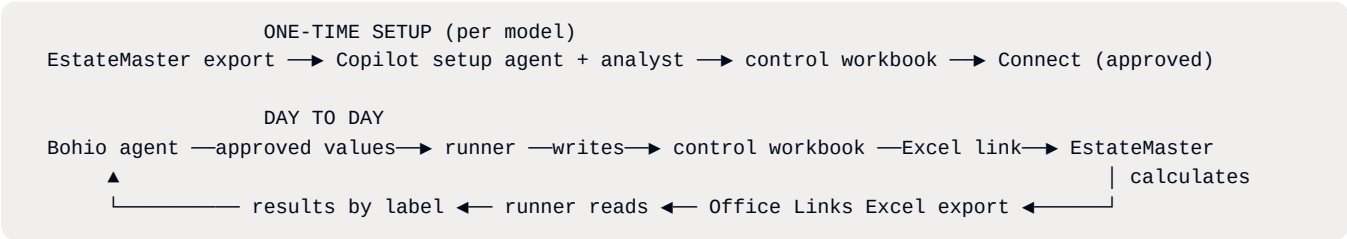
Setup guide

Copilot for one-time setup, the Bohio agent for day-to-day work:
Vercel, the EstateMaster runner, Copilot and Outlook

30 SEPTEMBER 2026 · REPLACES THE EARLIER SETUP GUIDE

EstateMaster has no API, so this setup runs everything through Excel and has a runner press EstateMaster's buttons. Nothing here relies on the EstateMaster manual: every step that depends on how EstateMaster behaves is marked "confirm in your trial". The runner has been tested end to end with a simulated EstateMaster, not yet with EstateMaster itself.

How it fits together



PIECE	WHERE IT RUNS	WHAT IT DOES
Bohio agent (the demo page)	Vercel	Chat, WhatsApp, proposals, approvals, Outlook scans, reports, change memory. Sends approved changes to the runner
<code>api/runner.js</code>	Vercel	Passes jobs to the runner; the runner token never reaches the browser
Runner (<code>runner/em_runner.py</code>)	Windows VM with EstateMaster and Excel	Writes the control workbook, operates EstateMaster, reads the export; hands a one-minute task to an analyst if automation fails
Control workbook	Runner VM (one per model)	The only place inputs are written. Inputs sheet: the 33 core lines in fixed cells (E4:E36). Lines sheet: every other assumption, no limit, found by id
KINAN master templates	Runner VM (<code>templates.json</code>)	Linked models new projects are copied from (mixed use, residential, retail)
Scratch copy and live model	Runner VM (<code>models.json</code>)	Scratch copy: where stress checks run and are undone. Live model: only written by an approved promotion, after a backup
Copilot setup agent	Microsoft 365 Copilot	Fills the control workbook for a new model from EstateMaster's export, with an analyst checking
Outlook reader (<code>api/scan.js</code>)	Vercel + Microsoft Graph	Reads one project folder four times a day and proposes assumption changes

EstateMaster is the trusted layer. Every figure the app computes itself is labelled as the agent's estimate. EstateMaster's figures come only from the runner's export, are labelled EstateMaster, and are tied to the exact working copy they were calculated for. "Check in EstateMaster" gets them at any time without changing the model. The AI never calculates.

Phases

PHASE	RESULT	TIME	WHO
0. Before you start	Accounts, machine and files ready	—	Bohio + KINAN IT
1. Deploy on Vercel	Agent online with Claude and/or OpenAI	30 min	Bohio
2. Prepare the EstateMaster trial	A model linked to a control workbook; round trip proven by hand	half a day	Bohio + analyst
3. Set up the runner	EstateMaster operated with no person at the keyboard, connected to the agent	1 day	Bohio
4. Copilot setup agent	A second model set up through Copilot and connected	2 hours	Analyst
5. Connect Outlook	Real email read and turned into proposals	1–2 hours	Microsoft 365 admin

Phase 0. Before you start

ITEM	NOTES
Windows VM (or PC) for the runner	Windows 10/11 or Server with desktop experience. Azure in a Saudi region, or KINAN's own. It must stay logged in (see 3.1)
EstateMaster (your trial first) on that machine	One pinned version. Trials may limit saving, export or Excel links confirm in your trial
Microsoft Excel (desktop) on that machine	For the control workbook and EstateMaster's Office Links
Python 3.11 or later on that machine	python.org installer, "Add to PATH" ticked
Vercel account	Free tier is enough to start
Anthropic and/or OpenAI API key	One is enough; both give failover
Cloudflare account (free) and a domain on it	For the tunnel that lets Vercel reach the runner without opening a port
Microsoft 365 Copilot licence	For each analyst doing setup (phase 4)
Microsoft 365 admin	For Outlook (phase 5) and to enable Copilot
The zip <code>bohio-estatemaster-vercel.zip</code>	<code>index.html</code> , <code>api/</code> , <code>runner/</code> , <code>setup/</code> , <code>docs/</code> , <code>README.md</code>

Ask Altus now (answers affect phases 2 and 3): Is unattended, automated use of EstateMaster on a VM allowed under KINAN's licence? Do Excel links survive copying a model? Can Office Links Refresh and Excel export be run without dialogs? Is there any command-line or batch option?

Phase 1. Deploy on Vercel

1. Unzip `bohio-estatemaster-vercel.zip`. You get the folder `bohio-estatemaster-demo`.
2. Install the Vercel command line (needs Node.js): `npm i -g vercel`. In the folder run `vercel`, log in, accept the defaults (framework: Other, no build command). Or push the folder to GitHub and import it at vercel.com/new.
3. In the Vercel project: **Settings** → **Environment Variables**, add the variables below for **Production**, then redeploy (`vercel --prod`). Variables only apply after a redeploy.
4. Open your site (the `https://...` address, not a file on your computer). The Router button top right should read "Router · AI: Claude" (or OpenAI). If it says "rules only", open it: it explains why.
5. If you set an access code, open Router, enter it under Access code and save.

VARIABLE	VALUE	NEEDED FOR
<code>ANTHROPIC_API_KEY</code>	your Anthropic key	Claude
<code>OPENAI_API_KEY</code>	your OpenAI key	OpenAI
<code>DEMO_PASSWORD</code>	any access code	Recommended: protects your AI credits and the runner
<code>RUNNER_URL</code> , <code>RUNNER_TOKEN</code>	added in phase 3	The EstateMaster runner
<code>MS_...</code> , <code>OUTLOOK_...</code>	added in phase 5	Outlook

The runner and the setup folder are not deployed to Vercel (`.vercelignore`). Router → Run diagnostics should show 21/21 checks passing.

Demo without a runner. Until phase 3 is done, the app simulates the runner and says so on every result (“EstateMaster (simulated runner)”), using the demo’s stand-in model. For a client demo you can show the analyst fallback: Architecture → Demo runner → “automation fails → analyst task”.

Phase 2. Prepare the EstateMaster trial

Goal: one model whose inputs come from the control workbook, and proof, by hand, that a change in the workbook changes EstateMaster’s results.

2.1 A model to work with

1. On the runner machine, open the trial and create or open a feasibility model. A copy of a real KINAN model is best; otherwise a simple mixed-use scheme like Al Narjis.
2. Save it as `C:\Bohio\Models\AlNarjis.emdf`. The agent only ever works on this copy, never on the live model.

2.2 The control workbook

1. Copy `setup\KINAN_control_workbook_template.xlsx` to `C:\Bohio\Models\AlNarjis_control.xlsx`.
2. On the **Inputs** sheet, fill column E with the model’s current values (by hand for this first model; phase 4 uses Copilot). Lines the model does not have: leave E blank and put N/A in column H. Put Y in column H for each line you have checked.
3. Do not insert or delete rows or columns or rename sheets: the runner writes to fixed cells (Inputs!E4:E36, listed in `runner/register_map.csv`).
4. Every other input you want the agent to reach goes on the **Lines** sheet: one row per input, no limit, id L0001, L0002, ... added at the bottom and never reused. Lines the runner may write need Y in column J once they are linked.
5. Check it: `py runner\control_check.py C:\Bohio\Models\AlNarjis_control.xlsx`. It must end with READY TO CONNECT.

2.3 Link the model to the workbook

1. In EstateMaster, link each input to its cell in column E using Office Links: the core lines on Inputs and each row on Lines, then put Y in column J of those rows. [confirm in your trial](#)
2. Save the model. Close Excel (the runner writes the workbook while it is closed).
3. Save a second copy of the model and check it still reads from the workbook. If links survive copying, this linked model becomes the **KINAN master template**: new projects start as copies of it and need no linking. [confirm in your trial](#)

2.4 Prove the round trip by hand, and time it

1. In the workbook, change MEP services (row 21) by +10%. Save and close Excel.
2. In EstateMaster: Office Links → Refresh, then recalculate. Note exactly which buttons you pressed and any dialog that appeared. [confirm in your trial](#)
3. Check the Summary tab: the IRR should change. Undo the change and refresh again: the IRR should return.
4. Office Links → Excel: export the Summary and Cash Flow. Open the export and note the exact row labels of Project IRR, Development Margin, Peak Debt and the others in `runner/output_labels.csv`, and whether IRR shows as 18.4 or 0.184. [confirm in your trial](#)
5. Time the whole round trip. Repeat with a timing input (delay) and a finance input (LTC).

Phase 3. Set up the runner

3.1 Prepare the machine

- Create a dedicated Windows user for the runner. Turn on automatic sign-in, and turn off screen lock, sleep and screen-saver: pressing buttons needs an unlocked desktop. On a VM, disconnecting Remote Desktop can lock the session; reconnect through the console or use a tool that keeps the session unlocked.
- Pin the EstateMaster version: turn off automatic updates. After any update, repeat 3.3 to 3.5.
- Install Python, then in PowerShell: `py -m pip install fastapi uvicorn openpyxl pywinauto pillow`.
- Copy the `runner` folder to `C:\Bohio\runner`.

3.2 Record EstateMaster's buttons

1. Run `py em_ui_probe.py C:\Bohio\Models\AlNarjis.emdf`. It opens the model and writes every button and tab it can see to `probe_out`, with screenshots.
2. When it pauses, open each dialog the round trip uses (Office Links → Excel export, Save As, any confirmation), press Enter to record it, and close it. Type q when done.
3. Edit `ui_steps.json`: for each group (refresh_links, recalculate, export_excel, save_copy, save_live, close), replace the placeholder names with the real ones from `probe_out`. `save_live` saves the model itself and is used only when promoting to the live model. Steps can click a button by title, press keys, type in a field or wait for a dialog. Add a step for every dialog you met in 2.4. 

3.3 Output labels

Edit `output_labels.csv`: put the exact row labels from your export (2.4) in the label column. The agent expects IRR and margins as percentages (18.4): if the export shows 0.184, set scale to 100 for that row.

3.4 The trial: one round trip with nobody at the keyboard

1. Run `py em_ui_trial.py C:\Bohio\Models\AlNarjis.emdf C:\Bohio\Models\AlNarjis_control.xlsx c4=1430`.
2. Do not touch the mouse or keyboard. It writes the value, opens the model, refreshes, recalculates, exports, saves a copy, closes and reads the results.
3. Read `trial_out\trial_....json`: "ok": true, the outputs, and the seconds per step. On failure it stops, saves a screenshot and reports no numbers; fix `ui_steps.json` and run again.
4. Pass: the IRR matches what you get doing the same change by hand, five runs in a row.

3.5 Connect models, templates, scratch copy and live model, and start the service

1. Copy `models.example.json` to `models.json` and list each model: its id, name, .emdf path and control workbook path. The id must match the project id in the app (the demo's Al Narjis project is `narjis`).
2. **Scratch copy** (for stress checks): save a second linked copy of the model and its workbook, e.g. `AlNarjis_scratch.emdf`, and add `scratch_emdf` and `scratch_control_workbook`. Without it, stress checks use the working model and the runner restores its workbook afterwards.
3. **Live model** (for promotion): add `live_emdf` and `live_control_workbook` for the model KINAN treats as live, linked the same way. Without them, promotions are refused. Every promotion first copies the live workbook to `work\backups`.
4. **Master templates** (for new models): copy `templates.example.json` to `templates.json` and point each entry (mixed, resi, retail) to a linked KINAN master model and its workbook. New models are copied from these into `work\models`; the structure of a new model is the template's.
5. Set the token and mode (new PowerShell windows pick them up): `setx RUNNER_TOKEN "<long random string>"` and `setx RUNNER_MODE "ui"`. Use "manual" until 3.4 passes reliably: every job then becomes a one-minute task for an analyst.
6. Start it: `py -m uvicorn em_runner:app --host 127.0.0.1 --port 8765`. To start it automatically, add a Task Scheduler task "at log on" of the runner user. It must run in the desktop session, not as a Windows service, because it presses buttons.

7. Test on the machine: `curl -H "Authorization: Bearer <token>" http://127.0.0.1:8765/health` shows the mode and connected models.

3.6 Let Vercel reach it (Cloudflare Tunnel)

1. Install `cloudflared` on the runner machine and create a named tunnel with a fixed hostname, e.g. `runner.bohio-kinan.com` → `http://localhost:8765` (Cloudflare Zero Trust → Networks → Tunnels). Quick tunnels change address on every restart: use them only for a first test.
2. Optional but recommended: put the hostname behind Cloudflare Access with a service token, and add `CF_ACCESS_CLIENT_ID` and `CF_ACCESS_CLIENT_SECRET` to Vercel.
3. In Vercel add `RUNNER_URL` (`https://runner.bohio-kinan.com`) and `RUNNER_TOKEN` (same as on the runner). Redeploy.
4. Check `https://<your-site>/api/runner`: “configured”: true, “reachable”: true. In the app, the header card and Router panel say “EstateMaster runner: connected”.

3.7 First end-to-end run from the app

1. In the app, go to Model data → **Connect control workbook** and pick the same control workbook. The app checks it and raises a change request. Connecting first makes the agent’s values and the workbook agree: the runner writes every line on every run.
2. Approve it in the Approvals tab. The runner writes the workbook and runs EstateMaster; the card shows “EstateMaster running...”, then EstateMaster’s IRR next to the agent’s estimate. Both go in the change memory.
3. Ask the agent a small change (“increase MEP by 5%”), approve it and check the result against EstateMaster by hand.
4. Press **Check in EstateMaster** under the returns on the Agent tab: EstateMaster’s own IRR for the working copy appears there, and the report’s EstateMaster column fills in.
5. Stress tests → Generate, then **Check worst 5 in EstateMaster**: they run on the scratch copy and it is restored after each one.
6. Promote a small change as an Investment Director and check the live model and the backup; create a test model from a template and check it appears in `models.json`.
7. If the card shows “waiting for an analyst”: do the task it names (open, Refresh, Export to the folder given), then press **Collect EstateMaster results**.

Phase 4. Copilot setup agent

From the second model on, analysts use Copilot to fill the control workbook. Full instructions and starter prompts are in `setup/copilot_setup_agent.md`.

4.1 Create it once

1. Confirm with KINAN IT which is enabled: **Copilot in Excel** (preferred: it can write into the workbook, the analyst accepts each edit) or a **Copilot agent** in Microsoft 365 Copilot (it returns a table the analyst pastes). Menu names change often.
2. Copilot in Excel: save the instructions from `copilot_setup_agent.md` where analysts can paste them as the first message. Copilot agent: create an agent, paste the instructions, add the SharePoint setup folder as knowledge, add the starter prompts.
3. Create the SharePoint setup folder with the control workbook template and `register_map.csv`.

4.2 Set up a model (analyst, about an hour)

1. In EstateMaster, Office Links → Excel on the model’s Input sheets; save the export in the SharePoint folder.
2. Copy the template as `<Project>_control.xlsx`, open Copilot and give it the export and the workbook: “Fill the Inputs sheet of this control workbook from the attached EstateMaster export.”
3. Review every line; put Y in column H for each one you agree with, N/A for lines the project does not use. Check totals (construction cost, GFA, land) against the EstateMaster Summary.
4. Run the checker, or in the app: Model data → Connect control workbook. Fix every error, read every warning.

5. Link the workbook to the model in EstateMaster (skip if the model is a copy of the KINAN master template).
confirm in your trial
6. Copy the model and workbook to the runner machine and add the model to `models.json` (Bohio, until this step is automated).
7. Connect it in the app. A second person approves the connection change request. The first approved change confirms the round trip.

Phase 5. Connect Outlook (Microsoft Graph)

The agent reads one mailbox folder, read-only; it never sends or deletes mail. Everything it finds becomes a change request.

1. Pick a project mailbox or folder (e.g. a shared mailbox `al-narjis@kinan.com`, or a folder "Al Narjis" under Inbox) and route project mail there.
2. Microsoft Entra admin center → **App registrations** → New registration ("Bohio agent – Outlook reader", this organisation only, no redirect URI). Copy the Application (client) ID and Directory (tenant) ID.
3. **API permissions** → Microsoft Graph → Application permissions → `Mail.Read` → **Grant admin consent**.
4. **Certificates & secrets** → New client secret; copy the value (shown once); note the expiry.
5. Restrict the app to the project mailbox (Exchange Online PowerShell, shown below) and test that other mailboxes are denied.
6. Add the variables below in Vercel and redeploy. `https://<your-site>/api/scan` should show "configured": true; the Outlook card says "Live".
7. Send a test email ("our MEP price moves to SAR 1,420 per sqm GFA"), press Scan now on the Outlook tab and approve the change request it raises. With the runner connected, EstateMaster recalculates.

```
Connect-ExchangeOnline
New-ApplicationAccessPolicy -AppId <client-id> -PolicyScopeGroupId bohio-mailboxes@kinan.com -AccessRight
RestrictAccess -Description "Bohio agent: project mailbox only"
Test-ApplicationAccessPolicy -Identity al-narjis@kinan.com -AppId <client-id> # Granted
Test-ApplicationAccessPolicy -Identity someone.else@kinan.com -AppId <client-id> # Denied
```

VARIABLE	VALUE
<code>MS_TENANT_ID</code>	Directory (tenant) ID
<code>MS_CLIENT_ID</code>	Application (client) ID
<code>MS_CLIENT_SECRET</code>	the secret value
<code>OUTLOOK_MAILBOX</code>	e.g. al-narjis@kinan.com
<code>OUTLOOK_FOLDER</code>	folder display name (default Inbox)
<code>EXTRACT_MODEL</code>	optional model override

Scans run at 06:00, 10:00, 14:00 and 18:00 Riyadh time while the workspace is open, and on request. Scans with nobody logged in need a Vercel Cron job and a database for proposals: not in the demo yet. Email content goes to the AI provider for extraction: confirm provider and region with KINAN compliance (Saudi PDPL).

Phase 6. Go live

- Altus has confirmed, in writing, unattended automated use under KINAN's licence.
- The trial round trip (3.4) passes five times in a row; `RUNNER_MODE` switched from manual to ui.
- The KINAN master template is linked and a copy keeps its links.

- Runner VM: dedicated user, automatic sign-in, no lock or sleep, EstateMaster version pinned, runner starts at log on, tunnel running, token stored securely.
- Access code set; API keys and the runner token only in Vercel and on the runner, never in the page.
- Outlook app restricted to the project mailbox and tested. Secret expiry in the calendar.
- Analysts trained with the user guide (docs/Bohio_EstateMaster_Agent_User_Guide.pdf). Nobody edits a live control workbook by hand.
- Router → Run diagnostics: 21/21 on the deployed site; runner and AI shown as connected.
- Promotion and new-model creation tested once each; live model backup restored once as a drill.

Running it

- Runner log: `C:\Bohio\runner\work\jobs.jsonl` (every job, inputs, outputs, timings). Failure screenshots are in the same folder.
- After an EstateMaster update: run the probe (3.2) and the trial (3.4) before switching back to ui mode. Meanwhile use manual mode.
- A new model: phase 4. A new output to report: add its export label to `output_labels.csv`.

Troubleshooting

SYMPTOM	LIKELY CAUSE AND FIX
Router says "rules only (no AI connected)"	Open Router: it says why (no key, not redeployed, opened from a file, Deployment Protection).
"EstateMaster runner: simulated"	RUNNER_URL or RUNNER_TOKEN missing in Vercel, or not redeployed. Expected until phase 3.
"configured but not reachable"	Runner not started, tunnel down, or Cloudflare Access blocking (check the CF_ACCESS variables).
"not connected on the runner"	The project id is not in the runner's models.json.
"model lines not linked in EstateMaster" / "not in the workbook"	The line is not marked Y in column J, or the runner's workbook is older than the one connected in the app: copy the current workbook to the runner.
"no live model configured" / "unknown template"	Add live_emdf and live_control_workbook to models.json, or the template to templates.json.
Returns card says the EstateMaster figure is for an earlier version	Something changed since EstateMaster last ran: press Check in EstateMaster.
Card stays at "waiting for an analyst"	Runner in manual mode, or automation failed (reason in the change memory). Do the task, then press Collect.
Automation fails at the same step	A button name changed or a new dialog appeared: re-run the probe and update ui_steps.json. Look at the failure screenshot.
"Not found in the export"	The label in output_labels.csv does not match the export exactly, or that sheet was not exported.
EstateMaster IRR does not change after a run	The link did not refresh: check 2.3/2.4 by hand; the workbook may have been open in Excel.
Connect control workbook shows errors	Rows moved, text in a value cell, percentages as 0.075, or missing values. Fix in the workbook and connect again.
"Microsoft sign-in failed" / "Graph 403"	Wrong IDs or expired secret / admin consent missing or mailbox not in the access policy.

Questions for Altus

- Is unattended, automated operation of EstateMaster on a VM allowed under KINAN's licence? Does it need a separate licence?
- Do Excel links survive copying a model? Can the linked workbook be repointed?
- Can Office Links Refresh and Excel export run without dialogs, or from a command line or macro?
- Will button names and dialogs change between versions? Is there advance notice of updates?
- Which inputs can be linked (unit mix, cost lines, timing, funding tranches)?

Appendix: files in the zip

FILE	PURPOSE
index.html	The Bohio agent (demo); Connect control workbook and runner status are on the Model data, Approvals and Architecture tabs
api/llm.js · api/scan.js · api/runner.js	AI proxy · Outlook reader · runner proxy (Vercel)
runner/em_runner.py	The runner service: queue, workbook writes, EstateMaster operation, fallback, Collect
runner/em_ui.py · em_ui_probe.py · em_ui_trial.py	Button presses · probe (3.2) · trial (3.4)
runner/ui_steps.json · output_labels.csv	Buttons to press · labels to read (placeholders until your trial)
runner/register_map.csv · control_check.py	Assumption register and fixed cells · workbook checker
runner/models.example.json · templates.example.json	Templates for models.json (with scratch and live) and templates.json (KINAN master templates)
setup/KINAN_control_workbook_template.xlsx	The control workbook (ReadMe, Project, Inputs, Mapping, Checks)
setup/copilot_setup_agent.md	Copilot instructions, starter prompts and analyst checks
setup/make_control_template.py	Regenerates the template and the fixed cells if the register changes
docs/	Features report, this guide, the user guide